

lab05

April 1, 2026

0.1 Sampling and reconstruction

```
[5]: from numpy import *
      from numpy.fft import ifft, fftfreq, fftshift
      from matplotlib.pyplot import *
```

0.1.1 Create the ‘continuous’ signal starting from a finite support Fourier transform

$$\text{triangle spectrum} \quad X(f) = \begin{cases} 1 - \frac{|f|}{f_{\max}} & |f| \leq f_{\max} \\ 0 & |f| > f_{\max} \end{cases}$$

$$x(t) = \mathcal{F}^{-1}[X(f)](t)$$

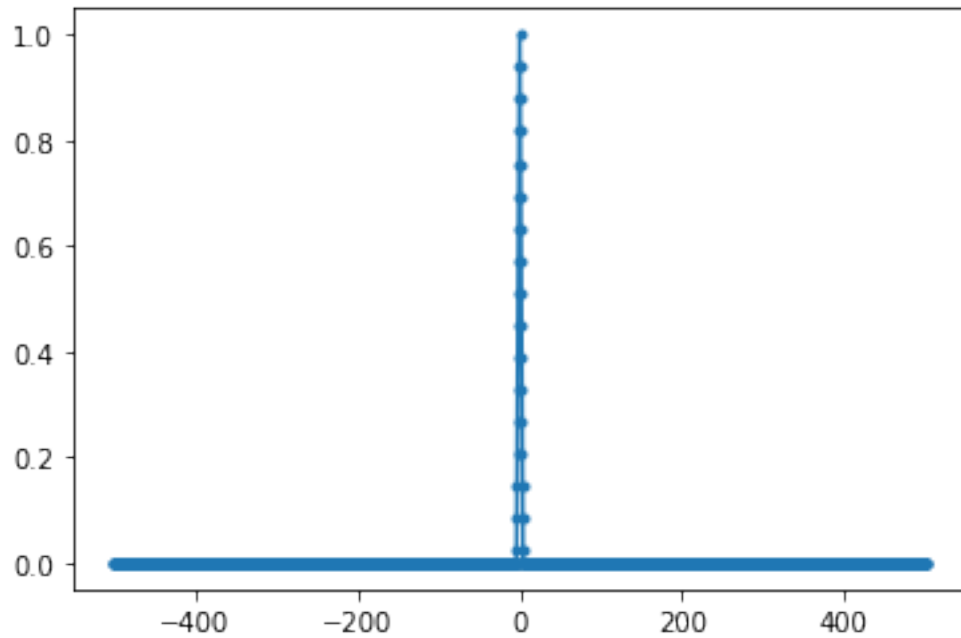
```
[16]: # We shall create a 'continuous' signal with a triangle shaped Fourier transform
      dt = 0.001
      fs = 1/dt # resolution of the 'continuous' signal

      fmax = 4.0 # maximum frequency (half width of the triangle)
      N = 4096 # the size of our 'continuous' signal: N points, i.e. N*T long
      f = fftfreq(N, dt) # the 'continuous' Fourier transform will have the same
      ↪ number of points at the same number of frequencies
      X = zeros(N)
      X[abs(f) <= fmax] = (1 - abs(f)/fmax)[abs(f) <= fmax] # triangle shape
      X # the peak is at index 0. Good!
```

```
[16]: array([1.          , 0.93896484, 0.87792969, ..., 0.81689453, 0.87792969,
        0.93896484])
```

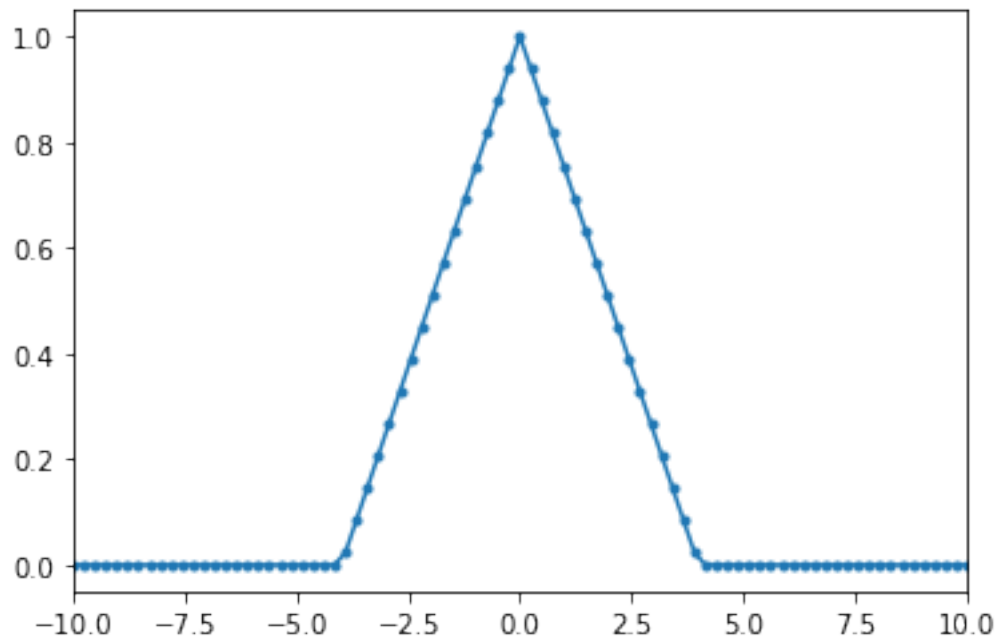
```
[17]: plot(fftshift(f), fftshift(X), '-')
```

```
[17]: [<matplotlib.lines.Line2D at 0x7f64cc5f6f40>]
```



```
[8]: plot(fftshift(f), fftshift(X), '.-')
      xlim([-10,10]) # zoom in to see better (non-interactive)
```

```
[8]: (-10.0, 10.0)
```

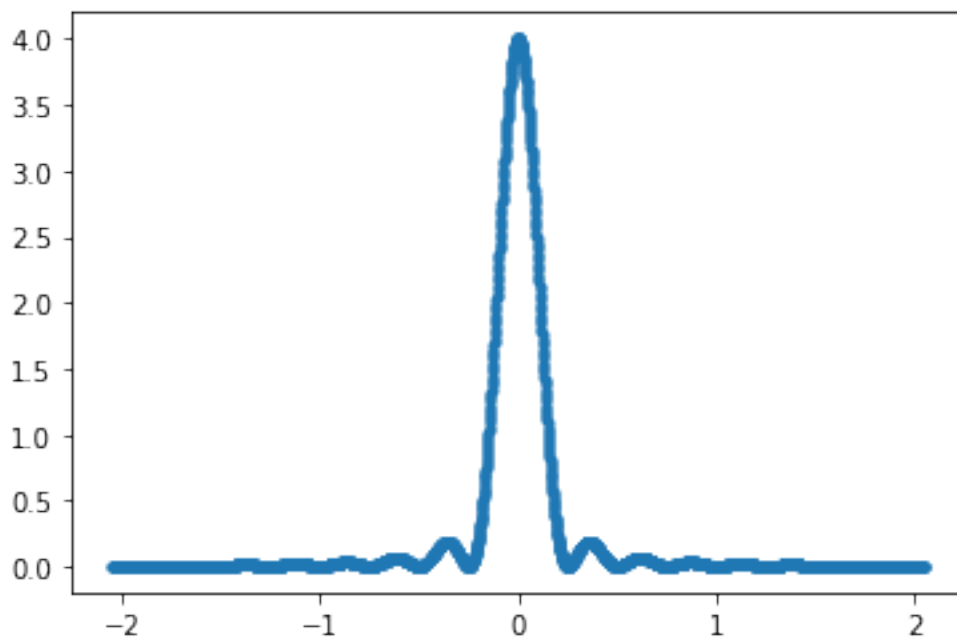


```
[9]: # get the signal as inverse Fourier transform
x = fs*real(ifft(X)) #  $N \cdot (fs/N) \cdot \text{ifft}(X)$ 
x
```

```
[9]: array([4.00352478, 4.00331389, 4.00268129, ..., 4.00162713, 4.00268129,
         4.00331389])
```

```
[10]: t = fftfreq(N, fs/N)
plot(fftshift(t), fftshift(x), '.-')
# Now we have our 'continuous' signal in x.
```

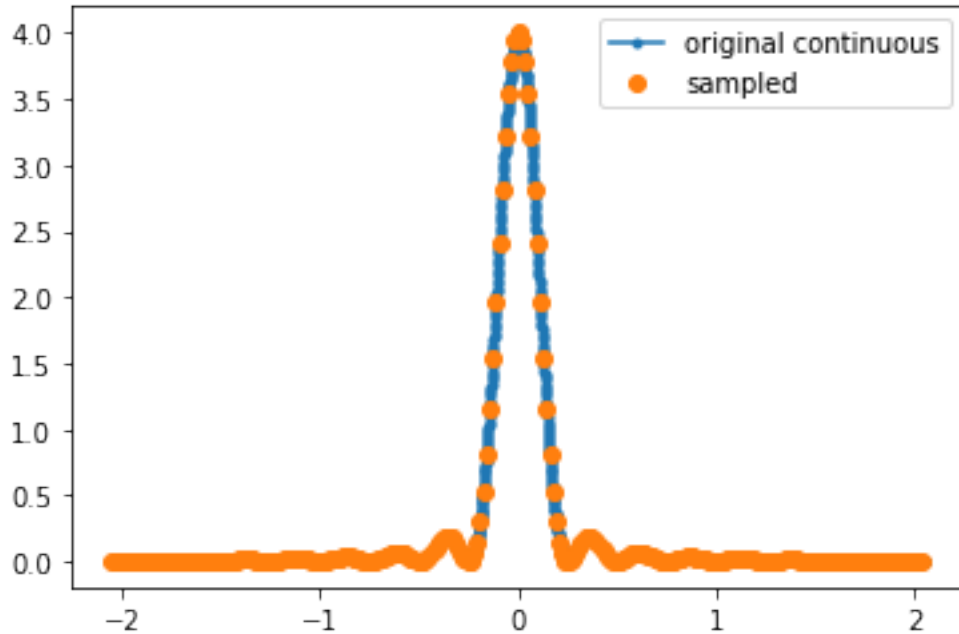
```
[10]: [<matplotlib.lines.Line2D at 0x7f64cca77280>]
```



0.1.2 Sampling

```
[21]: # Let's sample it by keeping only every 16th points
m = 16
T = dt*m # this is the sampling period
xhat = x[::m] # sampled (discrete time) signal
tsamp = t[::m] # sampled time
plot(fftshift(t), fftshift(x), '.-', label='original continuous')
plot(fftshift(tsamp), fftshift(xhat), 'o', label='sampled')
legend()
```

```
[21]: <matplotlib.legend.Legend at 0x7f64cc37cfa0>
```

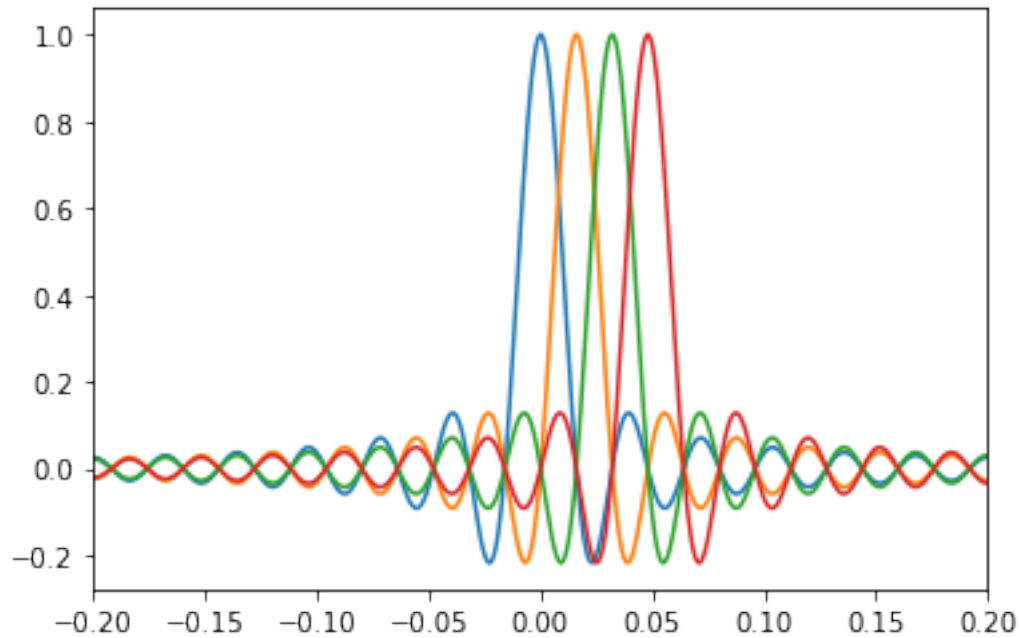


0.1.3 Reconstruction

$$x(t) = \mathcal{F}^{-1}[X](t) = \hat{x} * \frac{1}{T} \text{sinc}_{\pi}(t/T) = \sum_{n=-\infty}^{\infty} x[n] \frac{\sin \pi(t - nT)/T}{\pi(t - nT)/T} .$$

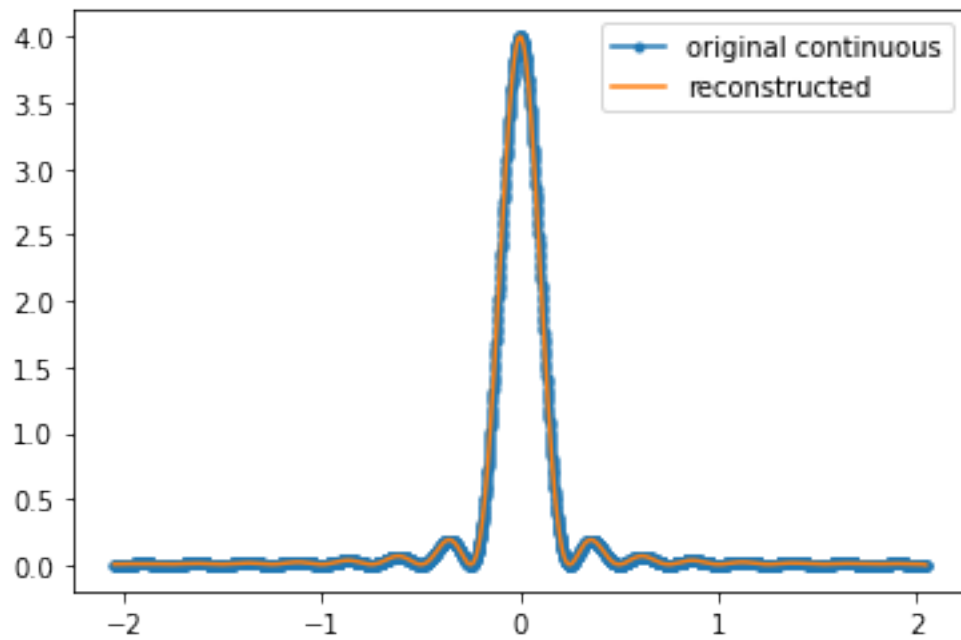
```
[19]: def sincpi(x):
    x[x == 0] = 1e-13 # to avoid problems when dividint with it
    return sin(pi*x)/(pi*x)
# these are the different shifted 'sinc' functions the linear combination of
# which reconstructs the original 'continuous' signal
plot(fftshift(t), fftshift(sincpi(t/T)))
plot(fftshift(t), fftshift(sincpi(t/T - 1)))
plot(fftshift(t), fftshift(sincpi(t/T - 2)))
plot(fftshift(t), fftshift(sincpi(t/T - 3)))
xlim([-0.2, 0.2])
```

[19]: (-0.2, 0.2)



```
[23]: Nsamp = N//m # size of the sampled signal (the same as len(xhat))
xrec = zeros(len(t)) # the reconstructed signal ('continuous')
for n in arange(-Nsamp//2, Nsamp//2):
    xrec += xhat[n]*sincpi(t/T - n)
plot(fftshift(t), fftshift(x), '.-', label='original continuous')
plot(fftshift(t), fftshift(xrec), '-', label='reconstructed')
legend()
```

[23]: <matplotlib.legend.Legend at 0x7f64cc2779d0>



[]: